Lab-5.4

Task Description #1:

• Prompt GitHub Copilot to generate a Python script that collects user data (e.g., name, age, email). Then, ask Copilot to add comments on how to anonymize or protect this data.

Expected Output #1:

• A script with inline Copilot-suggested code and comments explaining how to safeguard or anonymize user information (e.g., hashing emails, not storing data unencrypted)

```python
def collect_user_data():
    name = input("Enter your name: ")
    age = input("Enter your age: ")
    email = input("Enter your email: ")
    user_data = {
        "name": name,
        "age": age,
        "email": email
    }
    return user_data

def anonymize_user_data(user_data):
    # To anonymize user data:
    # - Do not store the real name; use a pseudonym or hash if needed
    # - Age can be stored as a range instead of exact value
    # - Email addresses should be hashed before storage to protect privacy

    import hashlib

    # Hash the email using SHA-256
    email_hash = hashlib.sha256(user_data["email"].encode()).hexdigest()

    # Replace name with a pseudonym (e.g., "User1")
    pseudonym = "User1"

    # Convert age to a range (e.g., 20-29, 30-39, etc.)
    try:
        age_int = int(user_data["age"])
        if age_int < 20:
            age_range = "<20"
        elif age_int < 30:
            age_range = "20-29"
        elif age_int < 40:
            age_range = "30-39"
```

```
            age_range = "<20"
        elif age_int < 30:
            age_range = "20-29"
        elif age_int < 40:
            age_range = "30-39"
        elif age_int < 50:
            age_range = "40-49"
        else:
            age_range = "50+"
    except ValueError:
        age_range = "Unknown"

    anonymized_data = {
        "name": pseudonym,
        "age_range": age_range,
        "email_hash": email_hash
    }
    return anonymized_data

if __name__ == "__main__":
    # Collect user data
    user_data = collect_user_data()
    print("Collected user data:", user_data)

    # Anonymize user data before storing or processing further
    anonymized = anonymize_user_data(user_data)
    print("Anonymized user data:", anonymized)
```

OUTPUT:

```
Enter your age: 19
Enter your email: rishitha@gmail.com
Collected user data: {'name': 'rishitha', 'age': '19', 'email': 'rishitha@gmail.com'}
Anonymized user data: {'name': 'User1', 'age_range': '<20', 'email_hash': 'b585b80506ac5a1deb96f7f726937dc122da6afcb3beb70f22c3334
44e71f40f'}
PS C:\Users\Rishitha Reddy> & "C:/Users/Rishitha Reddy/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/Rishitha Redd
y/OneDrive/Desktop/AIAC/Lab-5/task2.py"
Enter your name:
```

Task Description #2:

• Ask Copilot to generate a Python function for sentiment analysis. Then prompt Copilot to identify and handle potential biases in the data.

Expected Output #2:

• Copilot-generated code with additions or comments addressing bias mitigation strategies (e.g., balancing dataset, removing offensive terms).

C: > Users > Rishitha Reddy > OneDrive > Desktop > AIAC > Lab-5 > task1.2.py > ...

```python
def sentiment_analysis(word):
    """
    Simple sentiment analysis function.
    Returns 'positive', 'negative', or 'neutral' for a given word.
    This is a basic implementation and may not cover all edge cases.
    """
    # Example positive and negative word lists (expand as needed)
    positive_words = {"happy", "joy", "love", "excellent", "good", "great", "fantastic", "ama
    negative_words = {"sad", "hate", "bad", "terrible", "awful", "horrible", "angry", "disgus

    # Convert word to lowercase for case-insensitive matching
    word_lower = word.lower()

    # Bias Mitigation Strategies:
    # 1. Ensure the positive and negative word lists are balanced in size and diversity.
    # 2. Regularly review and update word lists to avoid cultural or social bias.
    # 3. Remove or flag potentially offensive or loaded terms from both lists.
    # 4. Consider using a larger, more representative dataset or pretrained models for produc

    if word_lower in positive_words:
        return "positive"
    elif word_lower in negative_words:
        return "negative"
    else:
        # For words not in the lists, return 'neutral'
        return "neutral"

# Example usage:
# print(sentiment_analysis("happy"))   # Output: positive
# print(sentiment_analysis("terrible")) # Output: negative
# print(sentiment_analysis("table"))    # Output: neutral

if __name__ == "__main__":
    import sys
```

```python
if __name__ == "__main__":
    import sys

    # If words are passed as command-line args, analyze each
    if len(sys.argv) > 1:
        for arg in sys.argv[1:]:
            print(sentiment_analysis(arg))
    else:
        # Interactive mode
        try:
            while True:
                user_input = input("Enter a word (or 'exit' to quit): ").strip()
                if user_input.lower() in {"exit", "quit", ""}:
                    break
                print(sentiment_analysis(user_input))
        except KeyboardInterrupt:
            pass
```

OUTPUT:

```
neutral
Enter a word (or 'exit' to quit): happy
positive
Enter a word (or 'exit' to quit): sad
negative
Enter a word (or 'exit' to quit): good
positive
Enter a word (or 'exit' to quit): excellent
positive
Enter a word (or 'exit' to quit):
```

Task Description #3:

• Use Copilot to write a Python program that recommends products based on user history. Ask it to follow ethical guidelines like transparency and fairness.

Expected Output #3:

• Copilot suggestions that include explanations, fairness checks (e.g., avoiding favoritism), and user feedback options in the code.

```
Product Recommendation System with Ethical Guidelines

- Transparency: The system explains why recommendations are made.
- Fairness: The system avoids favoritism and ensures diverse recommendations.
- User Feedback: Users can provide feedback to improve recommendations.
"""

def get_user_history():
    """
    Simulate fetching user purchase/view history.
    In a real system, this would come from a database.
    """
    # Example: user has interacted with electronics and books
    return ["laptop", "smartphone", "novel", "headphones"]

def get_product_catalog():
    """
    Simulate a product catalog.
    Each product has a category and a name.
    """
    return [
        {"name": "Laptop", "category": "Electronics"},
        {"name": "Smartphone", "category": "Electronics"},
        {"name": "Headphones", "category": "Electronics"},
        {"name": "Novel", "category": "Books"},
        {"name": "Cookbook", "category": "Books"},
        {"name": "Running Shoes", "category": "Sportswear"},
        {"name": "Yoga Mat", "category": "Sportswear"},
        {"name": "Board Game", "category": "Toys"},
        {"name": "Puzzle", "category": "Toys"},
    ]
```

```python
def recommend_products(user_history, catalog, num_recommendations=3):
    """
    Recommend products based on user history.
    - Recommends products from categories the user likes.
    - Ensures at least one product from a new category for diversity (fairness).
    - Explains the reason for each recommendation (transparency).
    """
    # Count categories in user history
    from collections import Counter
    history_categories = []
    for item in user_history:
        for product in catalog:
            if product["name"].lower() == item.lower():
                history_categories.append(product["category"])
    category_counts = Counter(history_categories)

    # Find top categories
    top_categories = [cat for cat, _ in category_counts.most_common()]

    # Recommend products from top categories, but ensure diversity
    recommended = []
    explained = []
    used_names = set([item.lower() for item in user_history])

    # 1. Recommend from top categories (not already interacted with)
    for cat in top_categories:
        for product in catalog:
            if product["category"] == cat and product["name"].lower() not in used_names:
                recommended.append(product)
                explained.append(
                    f"Recommended '{product['name']}' because you like {cat} products."
                )
                if len(recommended) >= num_recommendations - 1:
                    break
```

```python
def recommend_products(user_history, catalog, num_recommendations=3):
    # 1. Recommend from top categories (not already interacted with)
    for cat in top_categories:
        for product in catalog:
            if product["category"] == cat and product["name"].lower() not in used_names:
                recommended.append(product)
                explained.append(
                    f"Recommended '{product['name']}' because you like {cat} products."
                )
                if len(recommended) >= num_recommendations - 1:
                    break
        if len(recommended) >= num_recommendations - 1:
            break

    # 2. Add at least one product from a new category for fairness/diversity
    all_categories = set([p["category"] for p in catalog])
    new_categories = all_categories - set(top_categories)
    for cat in new_categories:
        for product in catalog:
            if product["category"] == cat and product["name"].lower() not in used_names:
                recommended.append(product)
                explained.append(
                    f"Recommended '{product['name']}' to introduce you to {cat} products for
                )
                break
        if len(recommended) >= num_recommendations:
            break

    # If not enough recommendations, fill with random products not in history
    if len(recommended) < num_recommendations:
        for product in catalog:
            if product["name"].lower() not in used_names and product not in recommended:
                recommended.append(product)
                explained.append(
```

```
                recommended.append(product)
                explained.append(
                    f"Recommended '{product['name']}' to introduce you to {cat} products for
                )
                break
        if len(recommended) >= num_recommendations:
            break

    # If not enough recommendations, fill with random products not in history
    if len(recommended) < num_recommendations:
        for product in catalog:
            if product["name"].lower() not in used_names and product not in recommended:
                recommended.append(product)
                explained.append(
                    f"Recommended '{product['name']}' to expand your options."
                )
            if len(recommended) >= num_recommendations:
                break

    return recommended[:num_recommendations], explained[:num_recommendations]

def get_user_feedback(recommended):
    """
    Allow user to provide feedback on recommendations.
    """
    print("\nWe value your feedback! Please rate the recommendations (1-5):")
    feedback = {}
    for product in recommended:
        while True:
            try:
                rating = int(input(f"How do you rate '{product['name']}'? (1=Bad, 5=Great): '
                if 1 <= rating <= 5:
                    feedback[product["name"]] = rating
```

```
            rating = int(input(f"How do you rate '{product['name']}'? (1=Bad, 5=Great): "))
            if 1 <= rating <= 5:
                feedback[product["name"]] = rating
                break
            else:
                print("Please enter a number between 1 and 5.")
        except ValueError:
            print("Please enter a valid number.")
    print("Thank you for your feedback! This will help us improve fairness and relevance.")

def main():
    print("Welcome to the Ethical Product Recommendation System!\n")
    user_history = get_user_history()
    catalog = get_product_catalog()
    print(f"Your recent activity: {', '.join(user_history)}")

    recommended, explanations = recommend_products(user_history, catalog)
    print("\nRecommended products for you:")
    for product, reason in zip(recommended, explanations):
        print(f"- {product['name']} ({product['category']}): {reason}")

    # Transparency: Show how recommendations were made
    print("\n[Transparency] Recommendations are based on your past interests and include div

    # User feedback
    get_user_feedback(recommended)

if __name__ == "__main__":
    main()
```

OUTPUT:

```
Your recent activity: laptop, smartphone, novel, headphones

Recommended products for you:
- Cookbook (Books): Recommended 'Cookbook' because you like Books products.
- Board Game (Toys): Recommended 'Board Game' to introduce you to Toys products for a diverse experie
nce.
- Running Shoes (Sportswear): Recommended 'Running Shoes' to introduce you to Sportswear products for
 a diverse experience.

[Transparency] Recommendations are based on your past interests and include diverse options to ensure
 fairness.

We value your feedback! Please rate the recommendations (1-5):
How do you rate 'Cookbook'? (1=Bad, 5=Great): 4
How do you rate 'Board Game'? (1=Bad, 5=Great): 5
How do you rate 'Running Shoes'? (1=Bad, 5=Great): 1
Thank you for your feedback! This will help us improve fairness and relevance.
PS C:\Users\Rishitha Reddy>
```

Task Description #4:

• Prompt Copilot to generate logging functionality in a Python web application. Then, ask it to ensure the logs do not record sensitive information.

Expected Output #4:

• Logging code that avoids saving personal identifiers (e.g., passwords, emails), and includes comments about ethical logging practices.

```
def is_valid_email(email: str) -> bool:
    """
    Basic email format validation.
    """
    email_pattern = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"
    return re.match(email_pattern, email) is not None


def prompt_email_or_password() -> None:
    """
    Ask the user whether they want to enter an email or a password, then prompt accordingly.
    Sensitive inputs are not logged; only non-sensitive metadata is recorded.
    """
    try:
        while True:
            choice = input("Do you want to enter 'email' or 'password'? ").strip().lower()
            if choice in {"email", "mail"}:
                email = input("Enter email: ").strip()
                is_valid = is_valid_email(email)
                print("Email format looks valid." if is_valid else "Invalid email format.")
                # Log only non-sensitive metadata
                log_user_action(
                    user_id="anonymous",
                    action="email_input",
                    details={"input_type": "email", "valid": is_valid}
                )
                break
            elif choice in {"password", "pass", "pwd"}:
                # Use getpass to avoid echoing the password
                try:
                    import getpass
                    password = getpass.getpass("Enter password: ")
                except Exception:
```

```
            is_strong_enough = len(password) >= 8
            print("Password received." + (" Consider using at least 8 characters." if not
            # Log only non-sensitive metadata
            log_user_action(
                user_id="anonymous",
                action="password_input",
                details={"input_type": "password", "strong_len": is_strong_enough}
            )
            break
        else:
            print("Please type 'email' or 'password'.")
except KeyboardInterrupt:
    print("\nCancelled by user.")


__name__ == "__main__":
# If launched with an argument, allow quick non-interactive selection
#   e.g., python task1.4.py email
#         python task1.4.py password
if len(sys.argv) > 1:
    arg = sys.argv[1].strip().lower()
    if arg in {"email", "mail"}:
        # Simulate the same flow directly
        email = input("Enter email: ").strip()
        is_valid = is_valid_email(email)
        print("Email format looks valid." if is_valid else "Invalid email format.")
        log_user_action(
            user_id="anonymous",
            action="email_input",
            details={"input_type": "email", "valid": is_valid}
```

```python
#         python task1.4.py password
if len(sys.argv) > 1:
    arg = sys.argv[1].strip().lower()
    if arg in {"email", "mail"}:
        # Simulate the same flow directly
        email = input("Enter email: ").strip()
        is_valid = is_valid_email(email)
        print("Email format looks valid." if is_valid else "Invalid email format.")
        log_user_action(
            user_id="anonymous",
            action="email_input",
            details={"input_type": "email", "valid": is_valid}
        )
    elif arg in {"password", "pass", "pwd"}:
        try:
            import getpass
            password = getpass.getpass("Enter password: ")
        except Exception:
            password = input("Enter password: ")
        is_strong_enough = len(password) >= 8
        print("Password received." + (" Consider using at least 8 characters." if not is_
        log_user_action(
            user_id="anonymous",
            action="password_input",
            details={"input_type": "password", "strong_len": is_strong_enough}
        )
    else:
        print("Unknown argument. Starting interactive mode...\n")
        prompt_email_or_password()
else:
    prompt_email_or_password()
```

OUTPUT:

```
Do you want to enter 'email' or 'password'? email
Enter email: rishitha@gmail.com
Email format looks valid.
2025-08-20 18:30:40,184 INFO UserAction | user_id=anonymous | action=email_input | details={'input_ty
pe': 'email', 'valid': True}
PS C:\Users\Rishitha Reddy> & "C:/Users/Rishitha Reddy/AppData/Local/Programs/Python/Python313/python
.exe" "c:/Users/Rishitha Reddy/OneDrive/Desktop/AIAC/Lab-5/task1.4.py"
Do you want to enter 'email' or 'password'? password
Enter password:
Password received.
2025-08-20 18:31:08,676 INFO UserAction | user_id=anonymous | action=password_input | details={'input
_type': 'password', 'strong_len': True}
PS C:\Users\Rishitha Reddy> & "C:/Users/Rishitha Reddy/AppData/Local/Programs/Python/Python313/python
.exe" "c:/Users/Rishitha Reddy/OneDrive/Desktop/AIAC/Lab-5/task1.4.py"
Do you want to enter 'email' or 'password'? email
Enter email: rishitha@gmail.com
Email format looks valid.
2025-08-20 18:32:48,663 INFO UserAction | user_id=anonymous | action=email_input | details={'input_ty
pe': 'email', 'valid': True}
PS C:\Users\Rishitha Reddy>
```

Task Description #5:

• Ask Copilot to generate a machine learning model. Then, prompt it to add documentation on how to use the model responsibly (e.g., explainability, accuracy limits).

Expected Output #5:

• Copilot-generated model code with a README or inline documentation suggesting responsible usage, limitations, and fairness considerations.

```python
def prompt_float(prompt_text: str) -> float:
    Prompt the user for a single floating-point number with validation.
    Re-prompts only if the input is invalid; returns once a valid float is entered
    """
    while True:
        try:
            value_str = input(prompt_text).strip()
            value = float(value_str)
            return value
        except ValueError:
            print("Please enter a valid number (e.g., 5.1).")

# Example usage:
if __name__ == "__main__":
    print("\nEnter iris flower features to predict its species.")
    print("Please provide the following measurements in centimeters (cm):")
    try:
        sepal_length = prompt_float("Enter sepal length (cm), e.g., 5.1: ")
        sepal_width = prompt_float("Enter sepal width (cm), e.g., 3.5: ")
        petal_length = prompt_float("Enter petal length (cm), e.g., 1.4: ")
        petal_width = prompt_float("Enter petal width (cm), e.g., 0.2: ")

        features = [sepal_length, sepal_width, petal_length, petal_width]
        species = predict
        print(f"Predicted
    except Exception as e:
```

5 / 5 Undo Keep

OUTPUT:

```
    print("Please provide the following measurements in centimeters (cm):")
    try:
        sepal_length = prompt_float("Enter sepal length (cm), e.g., 5.1: ")
        sepal_width = prompt_float("Enter sepal width (cm), e.g., 3.5: ")
        petal_length = prompt_float("Enter petal length (cm), e.g., 1.4: ")
        petal_width = prompt_float("Enter petal width (cm), e.g., 0.2: ")

        features = [sepal_length, sepal_width, petal_length, petal_width]
        species = predict_iris_species(features)
        print(f"Predicted species: {species}")
    except Exception as e:
        print("Invalid input. Please enter numeric values for all features.")

"""
Responsible AI Notice:
---------------------
- This model is for educational/demo purposes only.
- Always validate model performance on your own data before deployment.
- Consider the ethical implications of automated decision-making.
- For more explainability, consider visualizing the decision tree or using feature
"""
```

5 / 5 Undo Keep

Problems 3 Output Debug Console Terminal Ports Python

```
on/Python313/python.exe" "c:/Users/Rishitha Reddy/OneDrive/Desktop/AIAC/Lab-5/task1.5.py"
Please enter a valid number (e.g., 5.1).
Enter sepal length (cm), e.g., 5.1: 4.1
Enter sepal width (cm), e.g., 3.5: 7.8
Enter petal length (cm), e.g., 1.4:
```

Ctrl+K to generate a command